

todo-service -- Detailed Design Document

Version 1.0 · Status: Approved · Owner: Platform team Repository: adlc-poc-todo-service (CodeCommit, us-east-1)

1. Purpose and scope

todo-service is a small REST API for managing a personal todo list. It is intentionally minimal: a single Express process backed by an embedded SQLite database. This document is the source of truth for expected behavior. Any implementation that deviates from the contracts, invariants, or expected results below is defective.

2. Architecture overview

Single Node.js process, three modules, strict one-way dependency flow:

```
server.ts (HTTP layer: routing, input validation, status codes)
  |
  v
todos.ts  (domain layer: todo operations, row-to-DTO mapping)
  |
  v
db.ts     (persistence layer: SQLite connection, schema bootstrap)
```

Rules:

- server.ts never touches the database directly; it only calls todos.ts.
- todos.ts contains all SQL. SQL must ALWAYS use bound parameters (? placeholders).

String concatenation of user input into SQL is forbidden anywhere in the codebase -- this is a security invariant, not a style rule.

- db.ts owns the connection and schema. DB_PATH env var overrides the

database file location (default todos.db); tests use this to point at a throwaway file.

Technology: TypeScript 5.x (strict), Express 4, better-sqlite3 (synchronous), Node 22. Build is tsc to dist/; there is no bundler.

3. Data model

Table todos:

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
title	TEXT	NOT NULL
done	INTEGER	NOT NULL DEFAULT 0 (0 = open, 1 = done)
created_at	TEXT	NOT NULL DEFAULT datetime('now')

Row type (TodoRow in db.ts): { id: number; title: string; done: number; created_at: string }.

API DTO (Todo in todos.ts): { id: number; title: string; done: boolean; createdAt: string }. The mapping function toTodo(row) converts exactly one existing row; it must never be called with undefined.

4. Domain layer contract (todos.ts)

Function	Input	Expected result
<code>listTodos()</code>	--	All todos ordered by ascending id. Empty array when the table is empty.
<code>addTodo(title)</code>	non-empty string	Inserts the row, returns the created Todo with generated id, <code>done: false</code> .
<code>completeTodo(id)</code>	integer id	Marks the row done and returns the updated Todo. Returns <code>undefined</code> when no row has that id. Must not throw for a missing id.
<code>searchTodos(term)</code>	string	Todos whose title contains <code>term</code> (SQL LIKE, case-insensitive per SQLite default). Empty array when nothing matches. The term is passed as a bound parameter wrapped in <code>% . . %</code> .

Type-safety invariant: the codebase must compile with `npm run build` at all times. SQLite `get()` lookups are typed `TodoRow | undefined`; callers must narrow the optional before mapping (e.g. `row ? toTodo(row) : undefined`).

Security invariant (`searchTodos`): the SQL text must be a constant string such as `SELECT * FROM todos WHERE title LIKE ?` with the value `'%' + term + '%'` bound at execution. A search term containing quotes, percent signs, or SQL keywords must be treated as literal text to match, never as query structure.

5. HTTP API contract (server.ts)

All responses are JSON. Validation failures never reach the domain layer.

GET /todos

- 200 -> array of Todo, ordered by id. `[]` when empty.

POST /todos

- Body: `{ "title": string }`. Title is trimmed before insert.
- 201 -> the created Todo.
- 400 `{ "error": "title is required" }` when title is missing, not a string, or empty/whitespace-only.

POST /todos/:id/complete

- 200 -> the updated Todo with `done: true`. Completing an already-done todo is idempotent and still returns 200.
- 400 `{ "error": "invalid id" }` when `:id` is not an integer.
- 404 `{ "error": "not found" }` when no todo has that id.

GET /todos/search?q=term

- 200 -> array of matching Todos. `[]` for no matches.
- A missing `q` is treated as the empty string (matches every todo, since every title contains the empty string).
- Malicious values of `q` (e.g. `% ' OR '1'='1, ';' DROP TABLE todos; --`) must be matched literally and return `[]` unless a title actually contains that exact text. They must never alter the query, error out, or return unrelated rows.

6. Expected results -- worked examples

Starting from an empty database:

1. POST /todos {"title":"buy milk"} -> 201
{ "id":1, "title":"buy milk", "done":false, "createdAt":"<timestamp>" }
2. POST /todos {"title":"walk dog"} -> 201, id 2.
3. GET /todos -> 200, array of the two todos in id order.
4. POST /todos/1/complete -> 200, { "id":1, ..., "done":true }.
5. POST /todos/99/complete -> 404 { "error":"not found" }.
6. POST /todos/abc/complete -> 400 { "error":"invalid id" }.
7. GET /todos/search?q=milk -> 200, array containing only id 1.
8. GET /todos/search?q=%25%27%20OR%20%271%27%3D%271 (i.e. %' OR '1'='1')
-> 200 [] -- the payload is treated as literal text, not SQL.
9. POST /todos {"title":"" } -> 400 { "error":"title is required" }.

7. Unit test plan

Framework: Node's built-in test runner (node:test with node:assert/strict). Tests compile with the project (tsc) and run via npm test, which executes node --test dist/*.test.js. Each test file sets process.env.DB_PATH to a unique temp file BEFORE importing db/todos, and deletes it afterwards, so tests are isolated and repeatable.

todos.test.ts -- domain layer

#	Test	Arrange	Act	Assert
T1	listTodos empty	fresh db	listTodos()	returns []
T2	addTodo returns created todo	fresh db	addTodo("buy milk")	id >= 1, title "buy milk", done === false, createdAt non-empty string
T3	listTodos ordering	add 3 todos	listTodos()	3 items, ids strictly ascending
T4	completeTodo marks done	add one todo	completeTodo(id)	returns todo with done === true; listTodos() reflects it
T5	completeTodo missing id	fresh db	completeTodo(9999)	returns undefined, does not throw
T6	completeTodo idempotent	complete same id twice	second call	still returns todo with done === true
T7	searchTodos match	add "buy milk", "walk dog"	searchTodos("milk")	exactly one result, title "buy milk"
T8	searchTodos no match	same	searchTodos("zzz")	[]
T9	searchTodos empty term	same	searchTodos("")	returns all todos
T10	searchTodos injection literal	add "buy milk"	searchTodos("%' OR '1'='1'")	[] -- payload treated literally
T11	searchTodos quote handling	add todo titled it's done	searchTodos("it's")	exactly one result, no thrown error
T12	searchTodos wildcard literal?	add "100% done"	searchTodos("100% done")	returns the todo (LIKE wildcards in the term are acceptable to match broadly, but the call must not throw or leak other rows via injection)

server.test.ts -- HTTP layer (optional, via supertest or fetch against a listening port)

#	Test	Expect
H1	POST /todos valid	201 + created body
H2	POST /todos missing title	400 title is required
H3	POST /todos whitespace title	400
H4	POST /todos/:id/complete valid	200 done: true
H5	POST /todos/:id/complete unknown id	404
H6	POST /todos/abc/complete	400 invalid id
H7	GET /todos/search?q=injection payload	200 [], service still healthy afterwards (GET /todos works)

Acceptance gates

A change is acceptable only when ALL of the following hold:

1. `npm install && npm run build` exits 0 (TypeScript strict compile).
2. `npm test` exits 0 with every test above passing.
3. No SQL statement in the codebase concatenates user input.
4. Public API behavior in section 5 is unchanged for valid inputs.
5. The diff is minimal: only the code required for the fix, no refactoring, no dependency changes, no formatting churn.

8. Known non-goals

- No authentication (single-user PoC).
- No pagination, no delete endpoint, no persistence migrations.
- No `async/await` database layer (better-sqlite3 is synchronous by design).